MLP Optimization on Fashion MNIST: CPU → cuBLAS

Guilherme de Campos (guilhermedecampos12@gmail.com)
Mateus Dall'Igna (mateusdalligna.ifc@gmail.com)

Instituto Federal Catarinense (IFC) — Blumenau, SC, Brazil

> **Note:** this is the English translation of the original paper written in Brazilian Portuguese. The original is available at `docs/paper/ArtigoCUBLAS-pt.pdf`. Course paper, June 2026, Instituto Federal Catarinense (IFC).

Abstract

This paper presents a progressive optimization study of a multilayer perceptron neural network written from scratch in C++ for classification of the Fashion MNIST dataset. Starting from a sequential implementation running on a central processing unit, the model was successively optimized through OpenMP parallelization, hand-written CUDA functions with persistent memory, and finally the use of the cuBLAS library. The results show a speedup of up to 158.00x over the original implementation, reducing training time from 237.0 seconds to 1.5 seconds. In addition, an exploratory analysis was carried out to quantify the trade-off between execution time and accuracy through a relative efficiency metric.

Keywords

Neural Networks, CUDA, C++, OpenMP, cuBLAS, Optimization, Fashion MNIST

1 Introduction

The feasibility of training modern artificial neural networks is tied to the processing capacity of the underlying hardware. A multilayer perceptron is a feed-forward architecture composed of dense layers, where the data flow requires massive matrix operations. Mathematically, the pre-activation of each layer is defined by the linear equation $z = W \cdot x + b$, which is immediately followed by the non-linear post-activation given by $a = \sigma(z)$.

The computational bottleneck imposed by central processing units when training learning models is the main justification for this study. Moving the workload to highly parallel architectures, such as graphics processing units, has become the industry standard because of the matrix scalability of these components. Understanding the intermediate steps of software optimization — from parallelization on ordinary processors to the use of low-level linear algebra libraries — is therefore important for developing more efficient systems that operate at high performance.

In this context, the main goal of this work is to analyze, quantify and compare the performance gains obtained through four different iterative implementations of the same neural network architecture, applied to classification of the Fashion MNIST dataset.

To achieve this purpose, four essential objectives were defined. The first is to evaluate the effectiveness of processor parallelism using the OpenMP interface. The second focuses on measuring the impact of custom functions on the graphics card combined with persistent memory management. The third aims to compare the manual results with the implementation of the standard cuBLAS library. Finally, the fourth objective seeks to determine the ideal batch size that offers the best balance between convergence time and final accuracy of the statistical model, consolidating the computational cost-benefit analysis.

2 Theoretical background

Optimizing neural networks at the hardware level requires understanding parallel computing paradigms. While the machine's main processor is designed for low-latency sequential processing and complex control flow, graphics cards adopt the single-instruction multiple-thread architecture, with thousands of smaller cores designed for high throughput and independent matrix operations.

The CUDA platform acts as an interface that exposes the graphics card hardware for general-purpose computing. CUDA allows developers to write C++ functions that are processed simultaneously by thousands of parallel streams on the graphics card, abstracting the physical scheduling of cores. The technical literature shows that the biggest bottleneck when accelerating algorithms with CUDA is usually not the arithmetic processing itself, but the latency of data transfers between system memory and video memory over the motherboard bus. To mitigate this, optimized libraries such as cuBLAS are used by modern artificial intelligence ecosystems to perform linear algebra computations directly on blocks of data kept exclusively in the graphics card memory [3].

Within the scope of structural optimization of the network, research conducted by [7] identified a generalization gap phenomenon, proving that networks trained with very large batches tend to converge to sharp local minima in the loss function, which degrades the model's ability to infer unseen data. At the same time, [8] demonstrated the empirical benefits of training with smaller batches for better model performance, even though this underuses the massive parallelism of the hardware. The present work complements this literature by proposing a relative efficiency metric to find an optimal Pareto point considering the specific infrastructure of the test.

3 Dataset: Fashion MNIST

Fashion MNIST is a reference dataset for clothing image classification, created as a direct and more complex replacement for the classic MNIST. The dataset contains 70,000 grayscale images of 28 by 28 pixels, split into 60,000 training samples and 10,000 test samples, covering ten distinct categories (Table 1).

Table 1: Classes and their corresponding items in the Fashion MNIST dataset.

Class	Item	Class	Item
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

The choice of Fashion MNIST is justified by the saturation of the original handwritten digits dataset, where purely rudimentary statistical models already reach accuracies above 99.00%, masking the practical perception of architectural impacts and flaws.

To prepare the data, the raw pixel values were normalized to the continuous range between 0 and 1 by dividing them by the constant factor 255.0. The target vector of each sample was structured using one-hot encoding. All mathematical computations in this project were standardized to use 32-bit floating point precision.

4 Experimental methodology and architecture

The study evaluates four distinct implementations detailed in Table 2, keeping the same initial parametric conditions for a clean comparison isolated from the hardware factor.

Table 2: Summary of the tested implementations.

Number	Version	Technique applied
1	Sequential	Pure C++ code restricted to a single core
2	OpenMP	Parallelism on a multi-core processor
3	Manual CUDA	Custom functions on the graphics card
4	CUDA cuBLAS	Proprietary library optimized on the graphics card

4.1 Network configuration

The neural network structure used consists of three dense layers, totaling 109,386 trainable parameters. The two hidden layers use the rectified linear activation function, chosen for its computational efficiency in mitigating gradient dissipation. The output layer uses the sigmoid activation function.

The operating parameters listed in Table 3 were kept static across all performance runs to ensure a symmetric comparison. The initial learning rate was fixed at 0.1, a value adopted based on the methodology described in the study by He et al. [1]. Training was configured for 10 epochs, an amount empirically sufficient for the network to reach an initial learning plateau without overfitting the training data, thus offering a consistent timing window.

For weight initialization, the method proposed by the same methodology of He et al. [1] was adopted, which establishes a perfectly scaled variance to overcome the vanishing information inherent to linear activation functions. The learning algorithm uses the mean squared error as the loss metric, suitable for obtaining the continuous vector differential of the strict categorical encoding.

Table 3: Base parameters fixed for training.

Parameter	Configured value
Training epochs	10
Initial batch size	512
Learning rate	0.1
Loss function	Mean squared error
Weight initialization	He method
Bias initialization	Zeros

4.2 Experimental environment

To guarantee absolute reproducibility of the results and to contextualize the documented performance, Table 4 details the computational infrastructure used end to end.

Table 4: Specifications of the experimental environment.

Component	Specification
Operating system	EndeavourOS Linux 7.0.13
Central processor	AMD Ryzen 7 5700X3D with 8 cores
Processor memory	96 MiB of vertically stacked L3 cache
Graphics card	NVIDIA GeForce RTX 4060 Ti with 8 GB GDDR6
Graphics architecture	Ada Lovelace with compute capability 8.9
Graphics cores	4,352 CUDA processing cores
RAM	15 GiB of DDR4 memory
Compilers	GCC 16.1.1 and NVCC 13.3
Graphics driver version	610.43.02

The configuration of compiler flags is fundamental for low-level optimization. For the runs restricted to the central processor, the GCC compiler was invoked with maximum local vectorization attributes. Duration monitoring timed strictly the mathematical training loop using the language’s native library.

4.3 Evolution of the implementations

The methodology covered the successive coding of four evolutionary algorithms that solve the exact same neural network equation. The initial sequential version was implemented using matrices in contiguous memory blocks to optimize locality of fast access, and multipliers housed in simple loops.

Seeking to split the machine code across the main processor, the second iteration integrated the OpenMP interface [6]. Directives were inserted to slice the workload of large matrix processing among the sixteen hardware threads available.

The third iteration consisted of migrating the computational responsibility to the graphics card through entirely hand-written functions. To mitigate the transfer bottleneck on the computer’s main channel, a persistent memory architecture was adopted, in which the entirety of the weight and bias memory was transferred and pinned to the graphics card only once, later carrying only the sparse batches of input images and the raw numeric values returned by the loss verification.

In the final stage, the hand-written mathematical multipliers and adders were deleted from the codebase and replaced by native library routines from the graphics card vendor [3], which is calibrated for dense linear algebra manipulation directly on the graphics hardware.

5 Results and discussion

To ensure reliability and mitigate execution anomalies in the results, each of the four code solutions was tested an arbitrary number of 10 times in freshly initialized runs on the system. All convergence times and throughput measurements shown in this document represent the best values among those tests.

Validation of the performance leap generated by the different architectures used the mathematical concept of speedup factor, expressed as the direct ratio between the duration consumed by the elementary control version and the duration consumed by the optimized version being validated:

$$S = \frac{T_{base}}{T_{optimized}}$$

5.1 Performance comparison and speedup factor

Table 5 summarizes the data extracted from the four modalities, comparing them on an identical batch size of 512 images processed simultaneously. The sequential pass through the machine’s processor totaled 237.0 seconds. With the activation of core parallelism through OpenMP, the count dropped to 172.7 seconds, revealing a multiplicative advantage of 1.37x.

The rapid acceleration of the application emerged with the transfer of the mathematical workload to the graphics processor. The code written specifically by the developer on the graphics card finished all matrices in a mere 1.7 seconds, demonstrating an increase of 139.41x over the ordinary processor baseline. The version using the proprietary cuBLAS library showed the boundary mark of 1.5 seconds, formalizing a speedup of 158.00x.

Table 5: Comparative summary of the performance of the implemented versions considering ten runs.

Implemented version	Time	Throughput	Accuracy	Speedup
Sequential	237.0 s	2,532	87.7%	1.00x
OpenMP	172.7 s	3,475	87.2%	1.37x
Manual CUDA	1.7 s	344,850	79.7%	139.41x
CUDA cuBLAS	1.5 s	394,506	78.1%	158.00x

It is important to note that the solutions executed on the graphics card showed a regression in statistical accuracy compared with the main processor architectures, with a continuous drop in the range of 8.0 to 9.6 percentage points. This phenomenon occurs because of the reduction in the number of iterations performed, which, given the batch size, was deemed necessary for faster delivery.

5.2 Break-even analysis

The research continued with isolated trials focused on the cuBLAS version, with the goal of investigating which slice of data passed to the network produced the best learning. To price the simultaneous balance between training speed and network intelligence, the relative efficiency equation was defined based on the abstract concept of the optimal Pareto frontier, aimed at multi-objective optimization impasses [10]. The metric is structured as the product of the normalized accuracy range and the progressive reduction of processing time:

$$E(b) = \frac{A_b - A_{min}}{A_{max} - A_{min}} \times \frac{T_{max} - T_b}{T_{max} - T_{min}}$$

Here the variables denote the absolute time count and the hit percentage for a specific slice under analysis, with 87.5% accuracy set as the upper bound and 1.2 s taken as the speed limit. The fractional index immediately indicates how much the increase in batch population helps the graphics memory sweep without degrading the gradients of the computational network.

Table 6: Results of the batch-size experiment tied to the proposed efficiency metric.

Batch size	Time	Accuracy	Throughput	Final metric
128	2.5 s	87.5%	240,000	0.00
256	1.9 s	81.8%	321,000	0.39
512	1.5 s	79.1%	394,000	0.60
1024	1.3 s	73.9%	450,000	0.59
2048	1.2 s	61.8%	488,000	0.31
4096	1.2 s	50.1%	515,000	0.00

The data compilation shown in Table 6 brings to light the behavior previously seen in classical machine learning doctrine. For partitions with a population strictly below 512 individuals, the

mathematical scope no longer adequately feeds the logical clusters of the graphics card, depreciating computational power. Alternatively, excessively large groupings save only a few additional milliseconds of throughput but impose the cost of sacrificing 37 percentage points of the inferential model's statistical reasoning, reinforcing the generalization gap theory documented by [7]. In this context, the batch of 512 samples reached the maximum value of the metric with 0.60 practical efficiency, establishing itself empirically as the best operating point for the evaluated architecture.

6 Conclusion

The progression of optimizations documented in this paper demonstrated a substantial reduction in the training time of the artificial neural network built from scratch, going from 237.0 seconds in the sequential architecture to 1.5 seconds. This evidences a practical speedup of 158.00 times through the native insertion and instrumental adoption of the cuBLAS library, demonstrating the proficiency of graphics cards against the difficulties of linear algebra imposed by matrices.

The study showed that proper memory allocation and maintaining a persistent state on the graphics card are fundamental to exploiting the parallel processing capacity of contemporary architectures, a result demonstrated by the 139.41x speedup obtained in the manual implementation. In addition, identifying batch size 512 as the optimal point highlights the need to balance the hardware's time gains against the predictive degradation caused by excessively large batches, making it possible to build models that are fast yet reliable.

References

[1] He, K., Zhang, X., Ren, S., and Sun, J. 2015. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 1026-1034.

[2] Zalando Research. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. Available at: https://github.com/zalandoresearch/fashion-mnist.

[3] NVIDIA. 2024. cuBLAS Library User Guide. Available at: https://docs.nvidia.com/cuda/cublas/.

[4] NVIDIA. Tensor Core Operations. Available at: https://developer.nvidia.com/tensor-cores.

[5] NVIDIA. 2020. TensorFloat-32 in the A100 GPU. Available at: https://blogs.nvidia.com/blog/2020/05/14/tensorfloat-32-precision-format/.

[6] OpenMP Architecture Review Board. OpenMP Application Programming Interface. Available at: https://www.openmp.org/.

[7] Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. 2016. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. ICLR 2017.

[8] Masters, D., and Luschi, C. 2018. Revisiting Small Batch Training for Deep Neural Networks. arXiv:1804.07612.

[9] Smith, S. L., Kindermans, P. J., Ying, C., and Le, Q. V. 2017. Don't Decay the Learning Rate, Increase the Batch Size. ICLR 2018.

[10] Marler, R. T., and Arora, J. S. 2004. Survey of multi-objective optimization methods for engineering. Structural and Multidisciplinary Optimization, 26(6), 369-395.

Appendix A: Source-code repository

The complete source code with all implementations discussed in this paper, as well as the logical files and command transcripts used for the exhaustive extraction of the performance metrics covered throughout the research, is publicly available without restrictions for commercial or academic use in the GitHub repository, and can be accessed and replicated unrestrictedly through the following address: https://github.com/MateusDallIgna/mlp-optimizations-study